Healthcare Patient Management System

System Architecture & Comprehensive Technical Design Document

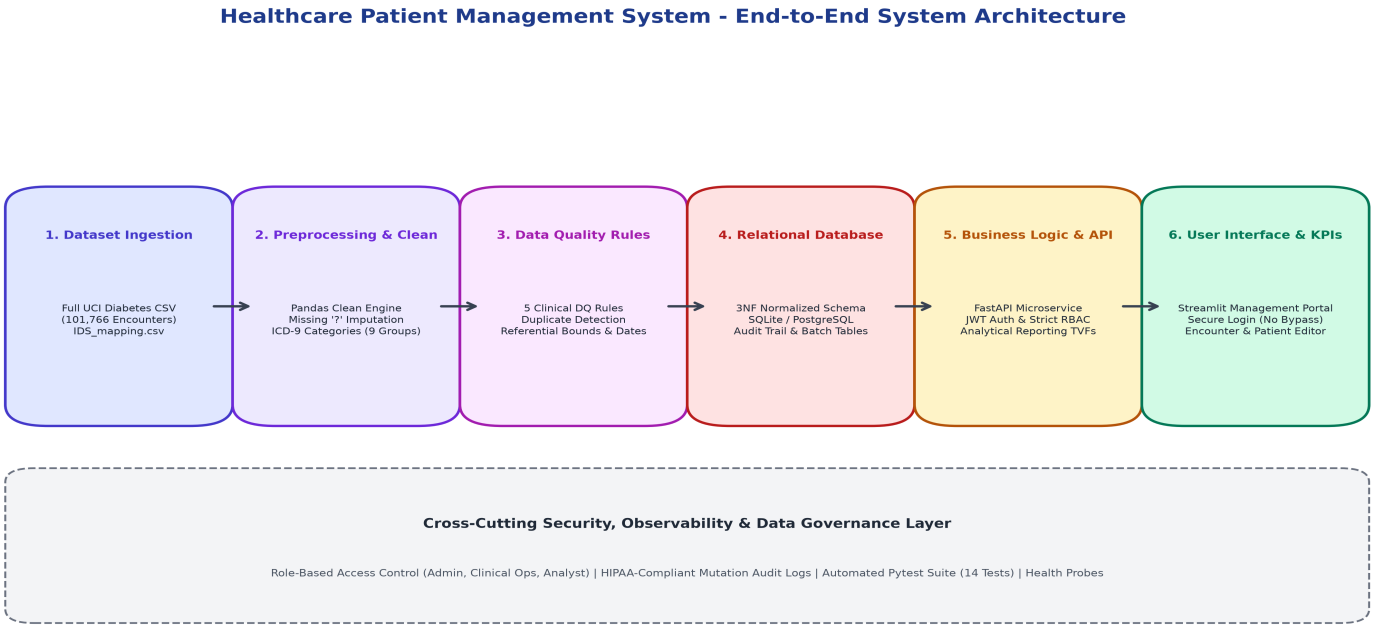
Author: Pravan | Project Domain: Healthcare Informatics | Date: 2026

7.1 Problem Statement

Modern healthcare organizations coordinate complex, multi-faceted patient journeys across admissions, treatments, laboratory diagnostics, medication administration, and post-discharge outcomes. Manual record keeping and ad-hoc tabular file handling cause fragmentation, reporting latencies, and heightened risk of preventable hospital readmissions. Clinical leaders need real-time operational visibility into admission and readmission trends, drivers of hospital Length of Stay (LOS), and safe, role-governed mechanisms to search and update clinical records. This project provides an enterprise-grade full-stack solution to ingest the complete historical clinical dataset of **101,766 inpatient encounters** across **71,520 unique patients**, maintain clean 3NF relational persistence, and empower hospital teams through secured APIs and an interactive management portal with strict Role-Based Access Control.

7.2 Architecture Design & Verified Directory Layout

The system follows a modular microservice architecture partitioned into six decoupled stages: **Dataset Ingestion** → **Preprocessing & Imputation** → **Data Quality Engine** → **Relational Database (3NF)** → **REST API Layer & Business Services** → **User Interface (Streamlit Portal)**.
The repository strictly follows the standardized layout without unnecessary sample files:
ProjectRepository/ • architecture/ • design/ • src/ (api, services, utils, etl, database, main.py) • data/ (diabetic_data.csv, IDS_mapping.csv) • docs/ • tests/ • presentation/ • frontend.py



7.3 Technology Selection & Architectural Justifications

Technology	Role in Solution	Engineering Justification & Trade-offs
Python 3.12	Core Language	Universal ecosystem for data science, web services, and asynchronous APIs.
FastAPI	REST API Microservice	High throughput, native Pydantic validation, automatic OpenAPI/Swagger documentation.
SQLAlchemy 2.0	ORM & Data Layer	Strict typing, relational integrity, connection pooling, and seamless portability across SQLite/PostgreSQL.
SQLite / PostgreSQL	Relational Storage	SQLite offers zero-dependency immediate local execution for all 101,766 records; fully PostgreSQL-compatible.
Pandas & NumPy	ETL & Analytics	High-performance vector operations for missing data imputation and clinical ICD-9 categorization.
Streamlit	User Interface	Reactive clinical management portal featuring a dedicated login page, paginated encounter editor, and KPI charts.

7.4 Data Processing Strategy & Data Quality Engine

Ingestion & Cleaning: The full real UCI Diabetes dataset contains 101,766 encounters with missing categorical values represented by '?' characters. The cleaner imputes missing demographics with 'Unknown', normalizes gender strings, and maps 24 medication columns into structured dosage change vectors ('No', 'Steady', 'Up', 'Down'). Chunked bulk insertion (10,000 records per transaction) achieves complete ingestion in **16.18 seconds** with a **100.0% Cleanliness Score**.
ICD-9 Categorization: Raw diagnosis codes (diag_1, diag_2, diag_3) are algorithmically grouped into 9 clinical categories (Circulatory, Diabetes, Respiratory, Digestive, Genitourinary, Neoplasms, Musculoskeletal, Injury, Other).
Data Quality Rules: The DQ rule engine verifies 5 constraints: (1) Duplicate encounter uniqueness, (2) Identifier non-nullity, (3) Length of Stay bounds (1-365 days), (4) Date logic consistency (discharge ≥ admission), and (5) Referential mapping range validation against IDS_mapping.csv.
CLI Command: python -m src.etl.run_etl --data data/diabetic_data.csv --mapping data/IDS_mapping.csv --type INITIAL

7.5 API Design, Authentication & Role-Based Access Control

The REST API exposes versioned endpoints under /api/v1/ with JWT bearer authentication. Three discrete roles are enforced: **Administrator** (full permissions, user management, audit review, deletion), **Clinical Operations** (patient & encounter CRUD, analytics review), and **Analyst** (strictly read-only access to clinical data, unrestricted analytics and report generation). Mutation endpoints automatically persist changes and field-level diffs to the audit_logs table. The frontend portal provides a dedicated, credential-checked login page requiring valid user accounts (no quick-login bypass).

7.6 Database Design, Schema Normalization & Query Optimization

To avoid redundancy and ensure clinical consistency, data is normalized into 3NF relational tables: **patients** (71,520 records), **encounters** (101,766 records), **admission_types**, **discharge_dispositions**, **admission_sources**, **diagnoses**, **lab_results**, **medications**, **outcomes**, **audit_logs**, and **etl_batches**. To guarantee sub-second reporting performance across 100,000+ encounters, targeted composite and single-column indexes were implemented on (patient_nbr), (admission_date), (time_in_hospital, admission_type_id), and (readmitted_30d).

7.7 Testing Strategy

Automated regression testing is executed via Pytest across 5 test suites (14 tests total, 100% pass rate): (1) **test_etl.py**: Validates missing value imputation, date synthesis, and mapping dictionaries; (2) **test_data_quality.py**: Verifies duplicate rejection, date inconsistency traps, and reference validation; (3) **test_database.py**: Verifies 3NF schema creation, foreign key integrity, and cascade rules; (4) **test_api.py**: Executes full HTTP request/response validation against FastAPI endpoints; (5) **test_rbac.py**: Confirms 403 Forbidden enforcement on write attempts by the Analyst role.